

# Deep Learning Mini Project

Dhairya Shah (ds6969)

Gunjan Dayani (gd2275)

Kevin Pattni (kbp2030)

## Importing required libraries

In [1]:

```
import tensorflow as tf
from keras.layers import Input, Conv2D, BatchNormalization, ReLU, Add, AveragePooling2D, Flatten, Dense
from keras.models import Model
from keras.datasets import cifar10
from keras.utils import to_categorical
from keras.preprocessing.image import ImageDataGenerator
from keras.optimizers import Adam

# Load the CIFAR-10 data
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>  
170498071/170498071 [=====] - 14s 0us/step

## Implementation of a ResNet Block

In [2]:

```
# Define the ResNet block
def residual_block(inputs, filters, strides):
    shortcut = inputs
    x = Conv2D(filters, kernel_size=3, strides=strides, padding="same")(inputs)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    x = Conv2D(filters, kernel_size=3, strides=1, padding="same")(x)
    x = BatchNormalization()(x)
    if strides != 1 or inputs.shape[-1] != filters:
        shortcut = Conv2D(filters, kernel_size=1, strides=strides)(inputs)
        shortcut = BatchNormalization()(shortcut)
    x = Add()([shortcut, x])
    x = ReLU()(x)
    return x
```

## Implementation of a ResNet architecture

In [3]:

```
# Define the ResNet architecture
inputs = Input(shape=(32, 32, 3))
x = Conv2D(64, kernel_size=3, strides=1, padding="same")(inputs)
x = BatchNormalization()(x)
x = ReLU()(x)
x = residual_block(x, filters=64, strides=1)
x = residual_block(x, filters=64, strides=1)
x = residual_block(x, filters=64, strides=1)
```

```
x = residual_block(x, filters=128, strides=2)
x = residual_block(x, filters=128, strides=1)
x = residual_block(x, filters=128, strides=1)
x = residual_block(x, filters=256, strides=2)
x = residual_block(x, filters=256, strides=1)
x = residual_block(x, filters=256, strides=1)
x = AveragePooling2D(pool_size=4)(x)
x = Flatten()(x)
outputs = Dense(10, activation="softmax")(x)
model = Model(inputs=inputs, outputs=outputs)
```

## Compiling our model

In [18]:

```
# Compile the model
opt = Adam(lr=0.001)
model.compile(optimizer=opt, loss="categorical_crossentropy", metrics=["accuracy"])
```

```
/usr/local/lib/python3.9/dist-packages/keras/optimizers/legacy/adam.py:117: UserWarning:
The `lr` argument is deprecated, use `learning_rate` instead.
  super().__init__(name, **kwargs)
```

## Training our ResNet model with some data augmentation

In [19]:

```
# Data augmentation
datagen = ImageDataGenerator(
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    rotation_range=10,
    zoom_range=0.1
)

# Train the model
batch_size = 128
epochs = 100
steps_per_epoch = x_train.shape[0] // batch_size
history = model.fit(datagen.flow(x_train, y_train, batch_size=batch_size),
                    steps_per_epoch=steps_per_epoch, epochs=epochs,
                    validation_data=(x_test, y_test))
```

```
Epoch 1/100
390/390 [=====] - 57s 139ms/step - loss: 0.4510 - accuracy: 0.84
48 - val_loss: 0.8001 - val_accuracy: 0.7314
Epoch 2/100
390/390 [=====] - 55s 141ms/step - loss: 0.3973 - accuracy: 0.86
27 - val_loss: 0.5170 - val_accuracy: 0.8297
Epoch 3/100
390/390 [=====] - 54s 139ms/step - loss: 0.3667 - accuracy: 0.87
41 - val_loss: 0.5210 - val_accuracy: 0.8281
Epoch 4/100
390/390 [=====] - 53s 136ms/step - loss: 0.3372 - accuracy: 0.88
37 - val_loss: 0.5286 - val_accuracy: 0.8313
Epoch 5/100
390/390 [=====] - 55s 140ms/step - loss: 0.3131 - accuracy: 0.89
22 - val_loss: 0.5393 - val_accuracy: 0.8305
Epoch 6/100
390/390 [=====] - 53s 135ms/step - loss: 0.2873 - accuracy: 0.90
20 - val_loss: 0.4690 - val_accuracy: 0.8505
Epoch 7/100
390/390 [=====] - 52s 134ms/step - loss: 0.2756 - accuracy: 0.90
42 - val_loss: 0.4534 - val_accuracy: 0.8582
Epoch 8/100
390/390 [=====] - 52s 133ms/step - loss: 0.2551 - accuracy: 0.91
06 - val_loss: 0.4993 - val_accuracy: 0.8444
Epoch 9/100
390/390 [=====] - 52s 133ms/step - loss: 0.2551 - accuracy: 0.91
06 - val_loss: 0.4993 - val_accuracy: 0.8444
```

```
390/390 [=====] - 52s 133ms/step - loss: 0.2384 - accuracy: 0.91
67 - val_loss: 0.4371 - val_accuracy: 0.8629
Epoch 10/100
390/390 [=====] - 55s 141ms/step - loss: 0.2265 - accuracy: 0.92
09 - val_loss: 0.6985 - val_accuracy: 0.8120
Epoch 11/100
390/390 [=====] - 54s 138ms/step - loss: 0.2110 - accuracy: 0.92
60 - val_loss: 0.5055 - val_accuracy: 0.8533
Epoch 12/100
390/390 [=====] - 55s 140ms/step - loss: 0.2017 - accuracy: 0.93
04 - val_loss: 0.3281 - val_accuracy: 0.8913
Epoch 13/100
390/390 [=====] - 55s 142ms/step - loss: 0.1932 - accuracy: 0.93
33 - val_loss: 0.4740 - val_accuracy: 0.8593
Epoch 14/100
390/390 [=====] - 53s 137ms/step - loss: 0.1820 - accuracy: 0.93
58 - val_loss: 0.4531 - val_accuracy: 0.8607
Epoch 15/100
390/390 [=====] - 56s 142ms/step - loss: 0.1726 - accuracy: 0.93
91 - val_loss: 0.3919 - val_accuracy: 0.8788
Epoch 16/100
390/390 [=====] - 56s 143ms/step - loss: 0.1603 - accuracy: 0.94
30 - val_loss: 0.4938 - val_accuracy: 0.8611
Epoch 17/100
390/390 [=====] - 53s 136ms/step - loss: 0.1514 - accuracy: 0.94
60 - val_loss: 0.3859 - val_accuracy: 0.8835
Epoch 18/100
390/390 [=====] - 56s 142ms/step - loss: 0.1462 - accuracy: 0.94
87 - val_loss: 0.4055 - val_accuracy: 0.8829
Epoch 19/100
390/390 [=====] - 54s 138ms/step - loss: 0.1383 - accuracy: 0.95
11 - val_loss: 0.4174 - val_accuracy: 0.8880
Epoch 20/100
390/390 [=====] - 54s 138ms/step - loss: 0.1356 - accuracy: 0.95
15 - val_loss: 0.3643 - val_accuracy: 0.8986
Epoch 21/100
390/390 [=====] - 55s 141ms/step - loss: 0.1200 - accuracy: 0.95
88 - val_loss: 0.3814 - val_accuracy: 0.8959
Epoch 22/100
390/390 [=====] - 54s 138ms/step - loss: 0.1182 - accuracy: 0.95
86 - val_loss: 0.4346 - val_accuracy: 0.8816
Epoch 23/100
390/390 [=====] - 54s 138ms/step - loss: 0.1136 - accuracy: 0.95
94 - val_loss: 0.5834 - val_accuracy: 0.8604
Epoch 24/100
390/390 [=====] - 55s 141ms/step - loss: 0.1096 - accuracy: 0.96
26 - val_loss: 0.4733 - val_accuracy: 0.8840
Epoch 25/100
390/390 [=====] - 55s 140ms/step - loss: 0.1029 - accuracy: 0.96
42 - val_loss: 0.5814 - val_accuracy: 0.8592
Epoch 26/100
390/390 [=====] - 55s 141ms/step - loss: 0.0993 - accuracy: 0.96
42 - val_loss: 0.4175 - val_accuracy: 0.8920
Epoch 27/100
390/390 [=====] - 55s 140ms/step - loss: 0.0921 - accuracy: 0.96
72 - val_loss: 0.3981 - val_accuracy: 0.8933
Epoch 28/100
390/390 [=====] - 53s 135ms/step - loss: 0.0923 - accuracy: 0.96
70 - val_loss: 0.3960 - val_accuracy: 0.8966
Epoch 29/100
390/390 [=====] - 55s 140ms/step - loss: 0.0901 - accuracy: 0.96
83 - val_loss: 0.5343 - val_accuracy: 0.8772
Epoch 30/100
390/390 [=====] - 54s 139ms/step - loss: 0.0864 - accuracy: 0.97
01 - val_loss: 0.5608 - val_accuracy: 0.8776
Epoch 31/100
390/390 [=====] - 54s 139ms/step - loss: 0.0803 - accuracy: 0.97
13 - val_loss: 0.3960 - val_accuracy: 0.9065
Epoch 32/100
390/390 [=====] - 54s 138ms/step - loss: 0.0825 - accuracy: 0.97
07 - val_loss: 0.3710 - val_accuracy: 0.9047
Epoch 33/100
390/390 [=====] - 55s 140ms/step - loss: 0.0754 - accuracy: 0.97
```

```
390/390 [=====] - 55s 140ms/step - loss: 0.0754 - accuracy: 0.97
36 - val_loss: 0.3926 - val_accuracy: 0.8934
Epoch 34/100
390/390 [=====] - 54s 138ms/step - loss: 0.0755 - accuracy: 0.97
28 - val_loss: 0.3955 - val_accuracy: 0.9013
Epoch 35/100
390/390 [=====] - 52s 133ms/step - loss: 0.0722 - accuracy: 0.97
50 - val_loss: 0.4379 - val_accuracy: 0.8996
Epoch 36/100
390/390 [=====] - 55s 141ms/step - loss: 0.0687 - accuracy: 0.97
52 - val_loss: 0.3823 - val_accuracy: 0.9062
Epoch 37/100
390/390 [=====] - 57s 146ms/step - loss: 0.0659 - accuracy: 0.97
69 - val_loss: 0.4338 - val_accuracy: 0.8965
Epoch 38/100
390/390 [=====] - 55s 141ms/step - loss: 0.0641 - accuracy: 0.97
72 - val_loss: 0.3987 - val_accuracy: 0.9049
Epoch 39/100
390/390 [=====] - 55s 141ms/step - loss: 0.0670 - accuracy: 0.97
64 - val_loss: 0.5555 - val_accuracy: 0.8703
Epoch 40/100
390/390 [=====] - 55s 140ms/step - loss: 0.0647 - accuracy: 0.97
71 - val_loss: 0.6922 - val_accuracy: 0.8588
Epoch 41/100
390/390 [=====] - 55s 141ms/step - loss: 0.0596 - accuracy: 0.97
90 - val_loss: 0.3906 - val_accuracy: 0.9048
Epoch 42/100
390/390 [=====] - 55s 141ms/step - loss: 0.0556 - accuracy: 0.98
02 - val_loss: 0.3730 - val_accuracy: 0.9150
Epoch 43/100
390/390 [=====] - 54s 137ms/step - loss: 0.0598 - accuracy: 0.97
90 - val_loss: 0.4946 - val_accuracy: 0.8943
Epoch 44/100
390/390 [=====] - 55s 141ms/step - loss: 0.0580 - accuracy: 0.98
04 - val_loss: 0.3973 - val_accuracy: 0.9031
Epoch 45/100
390/390 [=====] - 55s 141ms/step - loss: 0.0534 - accuracy: 0.98
12 - val_loss: 0.4328 - val_accuracy: 0.9013
Epoch 46/100
390/390 [=====] - 55s 140ms/step - loss: 0.0558 - accuracy: 0.98
12 - val_loss: 0.5320 - val_accuracy: 0.8884
Epoch 47/100
390/390 [=====] - 53s 135ms/step - loss: 0.0515 - accuracy: 0.98
15 - val_loss: 0.4089 - val_accuracy: 0.9046
Epoch 48/100
390/390 [=====] - 52s 134ms/step - loss: 0.0506 - accuracy: 0.98
29 - val_loss: 0.5206 - val_accuracy: 0.8914
Epoch 49/100
390/390 [=====] - 55s 142ms/step - loss: 0.0522 - accuracy: 0.98
16 - val_loss: 0.5068 - val_accuracy: 0.8928
Epoch 50/100
390/390 [=====] - 55s 140ms/step - loss: 0.0446 - accuracy: 0.98
40 - val_loss: 0.3996 - val_accuracy: 0.9099
Epoch 51/100
390/390 [=====] - 53s 137ms/step - loss: 0.0488 - accuracy: 0.98
24 - val_loss: 0.4335 - val_accuracy: 0.9085
Epoch 52/100
390/390 [=====] - 55s 141ms/step - loss: 0.0448 - accuracy: 0.98
50 - val_loss: 0.5081 - val_accuracy: 0.8936
Epoch 53/100
390/390 [=====] - 56s 144ms/step - loss: 0.0480 - accuracy: 0.98
27 - val_loss: 0.3793 - val_accuracy: 0.9104
Epoch 54/100
390/390 [=====] - 55s 140ms/step - loss: 0.0464 - accuracy: 0.98
38 - val_loss: 0.3955 - val_accuracy: 0.9104
Epoch 55/100
390/390 [=====] - 55s 142ms/step - loss: 0.0457 - accuracy: 0.98
46 - val_loss: 0.4519 - val_accuracy: 0.9036
Epoch 56/100
390/390 [=====] - 55s 142ms/step - loss: 0.0435 - accuracy: 0.98
52 - val_loss: 0.4252 - val_accuracy: 0.9080
Epoch 57/100
390/390 [=====] - 55s 141ms/step - loss: 0.0417 - accuracy: 0.98
```

```
390/390 [=====] - 55s 141ms/step - loss: 0.0417 - accuracy: 0.98
53 - val_loss: 0.4274 - val_accuracy: 0.9061
Epoch 58/100
390/390 [=====] - 53s 135ms/step - loss: 0.0396 - accuracy: 0.98
67 - val_loss: 0.4221 - val_accuracy: 0.9121
Epoch 59/100
390/390 [=====] - 55s 140ms/step - loss: 0.0393 - accuracy: 0.98
63 - val_loss: 0.4982 - val_accuracy: 0.9010
Epoch 60/100
390/390 [=====] - 54s 137ms/step - loss: 0.0398 - accuracy: 0.98
64 - val_loss: 0.4692 - val_accuracy: 0.8990
Epoch 61/100
390/390 [=====] - 54s 137ms/step - loss: 0.0398 - accuracy: 0.98
63 - val_loss: 0.4137 - val_accuracy: 0.9123
Epoch 62/100
390/390 [=====] - 55s 140ms/step - loss: 0.0402 - accuracy: 0.98
62 - val_loss: 0.3890 - val_accuracy: 0.9126
Epoch 63/100
390/390 [=====] - 54s 138ms/step - loss: 0.0363 - accuracy: 0.98
74 - val_loss: 0.3557 - val_accuracy: 0.9229
Epoch 64/100
390/390 [=====] - 53s 137ms/step - loss: 0.0357 - accuracy: 0.98
79 - val_loss: 0.4366 - val_accuracy: 0.9073
Epoch 65/100
390/390 [=====] - 55s 140ms/step - loss: 0.0371 - accuracy: 0.98
70 - val_loss: 0.4250 - val_accuracy: 0.9119
Epoch 66/100
390/390 [=====] - 55s 142ms/step - loss: 0.0360 - accuracy: 0.98
76 - val_loss: 0.4122 - val_accuracy: 0.9126
Epoch 67/100
390/390 [=====] - 55s 141ms/step - loss: 0.0365 - accuracy: 0.98
74 - val_loss: 0.4083 - val_accuracy: 0.9164
Epoch 68/100
390/390 [=====] - 53s 136ms/step - loss: 0.0361 - accuracy: 0.98
78 - val_loss: 0.4383 - val_accuracy: 0.9053
Epoch 69/100
390/390 [=====] - 54s 137ms/step - loss: 0.0341 - accuracy: 0.98
85 - val_loss: 0.4900 - val_accuracy: 0.9047
Epoch 70/100
390/390 [=====] - 55s 140ms/step - loss: 0.0347 - accuracy: 0.98
81 - val_loss: 0.5135 - val_accuracy: 0.9054
Epoch 71/100
390/390 [=====] - 55s 142ms/step - loss: 0.0340 - accuracy: 0.98
87 - val_loss: 0.3691 - val_accuracy: 0.9223
Epoch 72/100
390/390 [=====] - 56s 142ms/step - loss: 0.0335 - accuracy: 0.98
88 - val_loss: 0.5341 - val_accuracy: 0.8939
Epoch 73/100
390/390 [=====] - 55s 141ms/step - loss: 0.0343 - accuracy: 0.98
83 - val_loss: 0.4279 - val_accuracy: 0.9119
Epoch 74/100
390/390 [=====] - 53s 135ms/step - loss: 0.0344 - accuracy: 0.98
78 - val_loss: 0.4299 - val_accuracy: 0.9134
Epoch 75/100
390/390 [=====] - 54s 139ms/step - loss: 0.0303 - accuracy: 0.98
93 - val_loss: 0.3788 - val_accuracy: 0.9199
Epoch 76/100
390/390 [=====] - 55s 142ms/step - loss: 0.0258 - accuracy: 0.99
11 - val_loss: 0.4897 - val_accuracy: 0.9046
Epoch 77/100
390/390 [=====] - 55s 142ms/step - loss: 0.0334 - accuracy: 0.98
84 - val_loss: 0.4960 - val_accuracy: 0.9067
Epoch 78/100
390/390 [=====] - 54s 137ms/step - loss: 0.0298 - accuracy: 0.98
97 - val_loss: 0.4728 - val_accuracy: 0.9092
Epoch 79/100
390/390 [=====] - 55s 140ms/step - loss: 0.0310 - accuracy: 0.98
98 - val_loss: 0.4615 - val_accuracy: 0.9078
Epoch 80/100
390/390 [=====] - 54s 137ms/step - loss: 0.0273 - accuracy: 0.99
09 - val_loss: 0.4308 - val_accuracy: 0.9163
Epoch 81/100
390/390 [=====] - 55s 141ms/step - loss: 0.0288 - accuracy: 0.99
10 - val_loss: 0.4308 - val_accuracy: 0.9163
```

```

390/390 [=====] - 55s 141ms/step - loss: 0.0302 - accuracy: 0.98
96 - val_loss: 0.5590 - val_accuracy: 0.9066
Epoch 82/100
390/390 [=====] - 55s 140ms/step - loss: 0.0295 - accuracy: 0.99
07 - val_loss: 0.3897 - val_accuracy: 0.9210
Epoch 83/100
390/390 [=====] - 54s 139ms/step - loss: 0.0288 - accuracy: 0.98
97 - val_loss: 0.4084 - val_accuracy: 0.9178
Epoch 84/100
390/390 [=====] - 54s 139ms/step - loss: 0.0251 - accuracy: 0.99
14 - val_loss: 0.5611 - val_accuracy: 0.9018
Epoch 85/100
390/390 [=====] - 54s 138ms/step - loss: 0.0274 - accuracy: 0.99
06 - val_loss: 0.4920 - val_accuracy: 0.9097
Epoch 86/100
390/390 [=====] - 55s 140ms/step - loss: 0.0272 - accuracy: 0.99
04 - val_loss: 0.4962 - val_accuracy: 0.8986
Epoch 87/100
390/390 [=====] - 55s 141ms/step - loss: 0.0280 - accuracy: 0.99
00 - val_loss: 0.4624 - val_accuracy: 0.9134
Epoch 88/100
390/390 [=====] - 56s 142ms/step - loss: 0.0232 - accuracy: 0.99
18 - val_loss: 0.4884 - val_accuracy: 0.9071
Epoch 89/100
390/390 [=====] - 55s 140ms/step - loss: 0.0303 - accuracy: 0.99
00 - val_loss: 0.4435 - val_accuracy: 0.9130
Epoch 90/100
390/390 [=====] - 53s 135ms/step - loss: 0.0275 - accuracy: 0.99
03 - val_loss: 0.4328 - val_accuracy: 0.9183
Epoch 91/100
390/390 [=====] - 55s 141ms/step - loss: 0.0253 - accuracy: 0.99
14 - val_loss: 0.3918 - val_accuracy: 0.9225
Epoch 92/100
390/390 [=====] - 56s 142ms/step - loss: 0.0256 - accuracy: 0.99
10 - val_loss: 0.5359 - val_accuracy: 0.9077
Epoch 93/100
390/390 [=====] - 53s 135ms/step - loss: 0.0274 - accuracy: 0.99
05 - val_loss: 0.5295 - val_accuracy: 0.9034
Epoch 94/100
390/390 [=====] - 54s 139ms/step - loss: 0.0240 - accuracy: 0.99
13 - val_loss: 0.4051 - val_accuracy: 0.9188
Epoch 95/100
390/390 [=====] - 53s 136ms/step - loss: 0.0239 - accuracy: 0.99
20 - val_loss: 0.3965 - val_accuracy: 0.9252
Epoch 96/100
390/390 [=====] - 55s 141ms/step - loss: 0.0253 - accuracy: 0.99
16 - val_loss: 0.4328 - val_accuracy: 0.9136
Epoch 97/100
390/390 [=====] - 53s 136ms/step - loss: 0.0254 - accuracy: 0.99
14 - val_loss: 0.4924 - val_accuracy: 0.9090
Epoch 98/100
390/390 [=====] - 53s 136ms/step - loss: 0.0246 - accuracy: 0.99
16 - val_loss: 0.4318 - val_accuracy: 0.9162
Epoch 99/100
390/390 [=====] - 55s 141ms/step - loss: 0.0241 - accuracy: 0.99
24 - val_loss: 0.5456 - val_accuracy: 0.9004
Epoch 100/100
390/390 [=====] - 54s 139ms/step - loss: 0.0224 - accuracy: 0.99
23 - val_loss: 0.4401 - val_accuracy: 0.9188

```

**The accuracy reached ~ 92% after 100 epochs.**

## Visualization of ResNet model accuracy during training

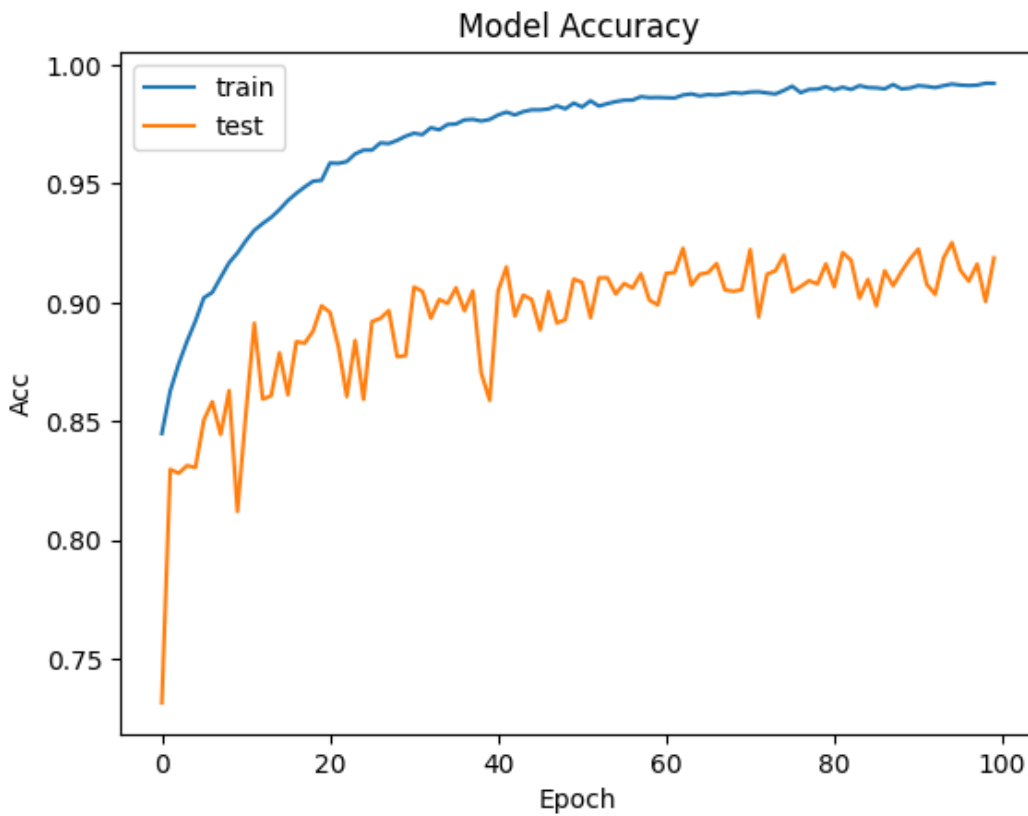
In [20]:

```

import matplotlib.pyplot as plt
#summarize history for accuracy
plt.plot(history.history['accuracy'])

```

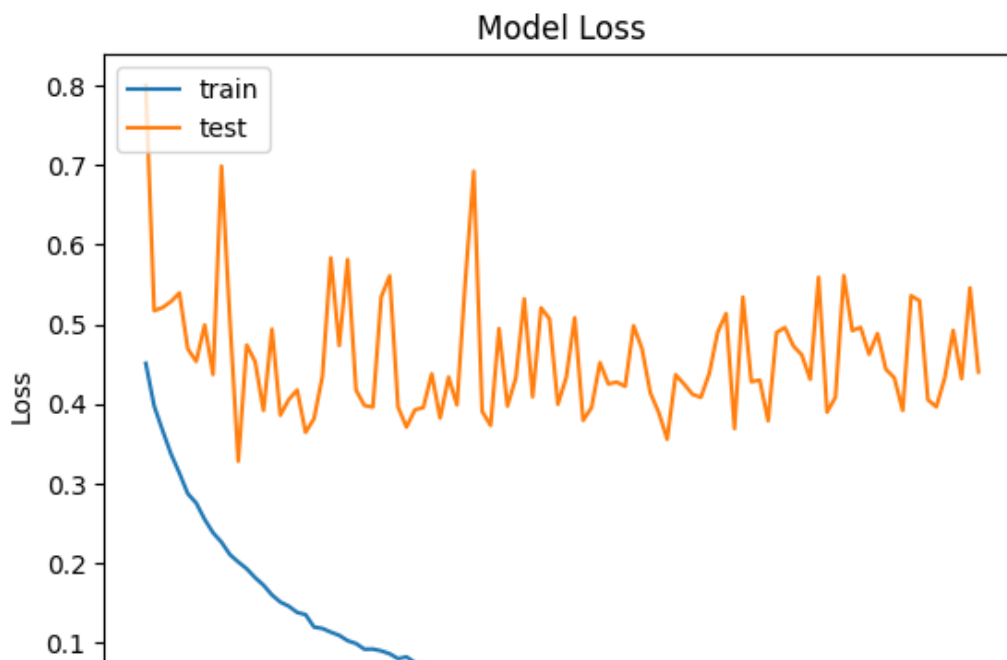
```
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Acc')
plt.xlabel('Epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

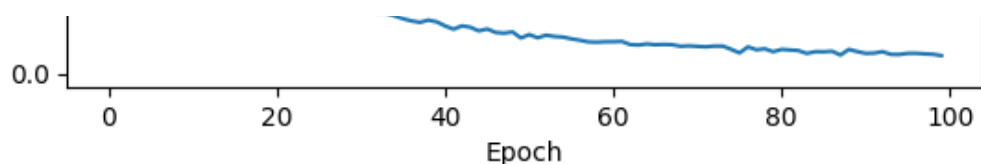


## Visualization of ResNet model loss during training

In [21]:

```
import matplotlib.pyplot as plt
#summarize history for accuracy
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```





## Visualizing and Evaluating Model Performance with Confusion Matrix and Classification Report

In [22]:

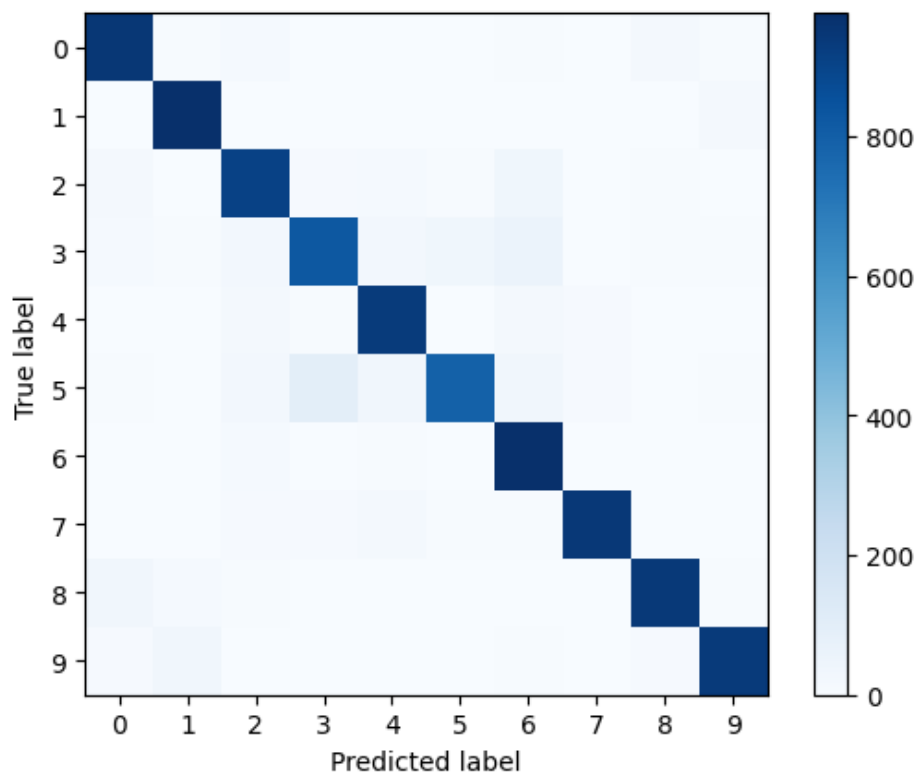
```
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

# Make predictions on the test data
y_pred = np.argmax(model.predict(x_test), axis=-1)

# Generate a confusion matrix
confusion_mtx = confusion_matrix(np.argmax(y_test, axis=-1), y_pred)
plt.imshow(confusion_mtx, interpolation='nearest', cmap=plt.cm.Blues)
plt.colorbar()
plt.xticks(range(10))
plt.yticks(range(10))
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.show()

# Generate a classification report
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
print(classification_report(np.argmax(y_test, axis=-1), y_pred, target_names=class_names
))
```

313/313 [=====] - 3s 10ms/step



|            | precision | recall | f1-score | support |
|------------|-----------|--------|----------|---------|
| airplane   | 0.91      | 0.95   | 0.93     | 1000    |
| automobile | 0.94      | 0.98   | 0.96     | 1000    |
| bird       | 0.89      | 0.91   | 0.90     | 1000    |
| cat        | 0.86      | 0.83   | 0.84     | 1000    |
| deer       | 0.91      | 0.94   | 0.92     | 1000    |
| dog        | 0.95      | 0.79   | 0.86     | 1000    |
| frog       | 0.85      | 0.98   | 0.91     | 1000    |



|              |      |      |      |       |
|--------------|------|------|------|-------|
| horse        | 0.97 | 0.94 | 0.96 | 1000  |
| ship         | 0.95 | 0.94 | 0.95 | 1000  |
| truck        | 0.96 | 0.94 | 0.95 | 1000  |
|              |      |      |      |       |
| accuracy     |      |      | 0.92 | 10000 |
| macro avg    | 0.92 | 0.92 | 0.92 | 10000 |
| weighted avg | 0.92 | 0.92 | 0.92 | 10000 |

## Evaluate our model and display test accuracy

In [23]:

```
# Evaluate the model on test set
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print("Test accuracy:", test_acc)
```

313/313 - 3s - loss: 0.4401 - accuracy: 0.9188 - 3s/epoch - 11ms/step  
Test accuracy: 0.9187999963760376

## Dsplaying the number of parameters used

In [24]:

```
import torchvision
from torchvision import models
model = Model(inputs=inputs, outputs=outputs)
print(model.summary())
```

Model: "model\_2"

| Layer (type)                                   | Output Shape        | Param # | Connected to                    |
|------------------------------------------------|---------------------|---------|---------------------------------|
| =====                                          |                     |         |                                 |
| input_1 (InputLayer)                           | [(None, 32, 32, 3)] | 0       | []                              |
| conv2d (Conv2D)                                | (None, 32, 32, 64)  | 1792    | ['input_1[0][0]']               |
| batch_normalization (BatchNorm<br>alization)   | (None, 32, 32, 64)  | 256     | ['conv2d[0][0]']                |
| re_lu (ReLU)                                   | (None, 32, 32, 64)  | 0       | ['batch_normalization[0][0]']   |
| conv2d_1 (Conv2D)                              | (None, 32, 32, 64)  | 36928   | ['re_lu[0][0]']                 |
| batch_normalization_1 (BatchNo<br>rmalization) | (None, 32, 32, 64)  | 256     | ['conv2d_1[0][0]']              |
| re_lu_1 (ReLU)                                 | (None, 32, 32, 64)  | 0       | ['batch_normalization_1[0][0]'] |
| conv2d_2 (Conv2D)                              | (None, 32, 32, 64)  | 36928   | ['re_lu_1[0][0]']               |

|                                             |                    |       |                                                     |
|---------------------------------------------|--------------------|-------|-----------------------------------------------------|
| batch_normalization_2 (Batch Normalization) | (None, 32, 32, 64) | 256   | ['conv2d_2[0][0]']                                  |
| add (Add)                                   | (None, 32, 32, 64) | 0     | ['re_lu[0][0]',<br>'batch_normalization_2[0][0]']   |
| re_lu_2 (ReLU)                              | (None, 32, 32, 64) | 0     | ['add[0][0]']                                       |
| conv2d_3 (Conv2D)                           | (None, 32, 32, 64) | 36928 | ['re_lu_2[0][0]']                                   |
| batch_normalization_3 (Batch Normalization) | (None, 32, 32, 64) | 256   | ['conv2d_3[0][0]']                                  |
| re_lu_3 (ReLU)                              | (None, 32, 32, 64) | 0     | ['batch_normalization_3[0][0]']                     |
| conv2d_4 (Conv2D)                           | (None, 32, 32, 64) | 36928 | ['re_lu_3[0][0]']                                   |
| batch_normalization_4 (Batch Normalization) | (None, 32, 32, 64) | 256   | ['conv2d_4[0][0]']                                  |
| add_1 (Add)                                 | (None, 32, 32, 64) | 0     | ['re_lu_2[0][0]',<br>'batch_normalization_4[0][0]'] |
| re_lu_4 (ReLU)                              | (None, 32, 32, 64) | 0     | ['add_1[0][0]']                                     |
| conv2d_5 (Conv2D)                           | (None, 32, 32, 64) | 36928 | ['re_lu_4[0][0]']                                   |
| batch_normalization_5 (Batch Normalization) | (None, 32, 32, 64) | 256   | ['conv2d_5[0][0]']                                  |
| re_lu_5 (ReLU)                              | (None, 32, 32, 64) | 0     | ['batch_normalization_5[0][0]']                     |
| conv2d_6 (Conv2D)                           | (None, 32, 32, 64) | 36928 | ['re_lu_5[0][0]']                                   |
| batch_normalization_6 (Batch Normalization) | (None, 32, 32, 64) | 256   | ['conv2d_6[0][0]']                                  |

|                                                      |                     |        |                                                                   |  |
|------------------------------------------------------|---------------------|--------|-------------------------------------------------------------------|--|
| rmalization)                                         |                     |        |                                                                   |  |
| add_2 (Add)                                          | (None, 32, 32, 64)  | 0      | ['re_lu_4[0][0]',<br>'batch_normalization_6[0][0]']               |  |
| re_lu_6 (ReLU)                                       | (None, 32, 32, 64)  | 0      | ['add_2[0][0]']                                                   |  |
| conv2d_7 (Conv2D)                                    | (None, 16, 16, 128) | 73856  | ['re_lu_6[0][0]']                                                 |  |
| batch_normalization_7 (BatchNormal<br>rmalization)   | (None, 16, 16, 128) | 512    | ['conv2d_7[0][0]']                                                |  |
| re_lu_7 (ReLU)<br>[0][0]']                           | (None, 16, 16, 128) | 0      | ['batch_normalization_7[0][0]']                                   |  |
| conv2d_9 (Conv2D)                                    | (None, 16, 16, 128) | 8320   | ['re_lu_6[0][0]']                                                 |  |
| conv2d_8 (Conv2D)                                    | (None, 16, 16, 128) | 147584 | ['re_lu_7[0][0]']                                                 |  |
| batch_normalization_9 (BatchNormal<br>rmalization)   | (None, 16, 16, 128) | 512    | ['conv2d_9[0][0]']                                                |  |
| batch_normalization_8 (BatchNormal<br>rmalization)   | (None, 16, 16, 128) | 512    | ['conv2d_8[0][0]']                                                |  |
| add_3 (Add)<br>[0][0]',<br>8[0][0]']                 | (None, 16, 16, 128) | 0      | ['batch_normalization_9[0][0]',<br>'batch_normalization_8[0][0]'] |  |
| re_lu_8 (ReLU)                                       | (None, 16, 16, 128) | 0      | ['add_3[0][0]']                                                   |  |
| conv2d_10 (Conv2D)                                   | (None, 16, 16, 128) | 147584 | ['re_lu_8[0][0]']                                                 |  |
| batch_normalization_10 (BatchNormal<br>ormalization) | (None, 16, 16, 128) | 512    | ['conv2d_10[0][0]']                                               |  |
| re_lu_9 (ReLU)<br>0[0][0]']                          | (None, 16, 16, 128) | 0      | ['batch_normalization_10[0][0]']                                  |  |

|                                              |                     |        |                                                       |
|----------------------------------------------|---------------------|--------|-------------------------------------------------------|
| conv2d_11 (Conv2D)                           | (None, 16, 16, 128) | 147584 | ['re_lu_9[0][0]']                                     |
| batch_normalization_11 (Batch Normalization) | (None, 16, 16, 128) | 512    | ['conv2d_11[0][0]']                                   |
| add_4 (Add)                                  | (None, 16, 16, 128) | 0      | ['re_lu_8[0][0]',<br>'batch_normalization_11[0][0]']  |
| re_lu_10 (ReLU)                              | (None, 16, 16, 128) | 0      | ['add_4[0][0]']                                       |
| conv2d_12 (Conv2D)                           | (None, 16, 16, 128) | 147584 | ['re_lu_10[0][0]']                                    |
| batch_normalization_12 (Batch Normalization) | (None, 16, 16, 128) | 512    | ['conv2d_12[0][0]']                                   |
| re_lu_11 (ReLU)                              | (None, 16, 16, 128) | 0      | ['batch_normalization_12[0][0]']                      |
| conv2d_13 (Conv2D)                           | (None, 16, 16, 128) | 147584 | ['re_lu_11[0][0]']                                    |
| batch_normalization_13 (Batch Normalization) | (None, 16, 16, 128) | 512    | ['conv2d_13[0][0]']                                   |
| add_5 (Add)                                  | (None, 16, 16, 128) | 0      | ['re_lu_10[0][0]',<br>'batch_normalization_13[0][0]'] |
| re_lu_12 (ReLU)                              | (None, 16, 16, 128) | 0      | ['add_5[0][0]']                                       |
| conv2d_14 (Conv2D)                           | (None, 8, 8, 256)   | 295168 | ['re_lu_12[0][0]']                                    |
| batch_normalization_14 (Batch Normalization) | (None, 8, 8, 256)   | 1024   | ['conv2d_14[0][0]']                                   |
| re_lu_13 (ReLU)                              | (None, 8, 8, 256)   | 0      | ['batch_normalization_14[0][0]']                      |
| conv2d_16 (Conv2D)                           | (None, 8, 8, 256)   | 33024  | ['re_lu_12[0][0]']                                    |

|                                              |                   |        |                                                                     |
|----------------------------------------------|-------------------|--------|---------------------------------------------------------------------|
| conv2d_15 (Conv2D)                           | (None, 8, 8, 256) | 590080 | ['re_lu_13[0][0]']                                                  |
| batch_normalization_16 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_16[0][0]']                                                 |
| batch_normalization_15 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_15[0][0]']                                                 |
| add_6 (Add)<br>6[0][0]',<br>15[0][0]']       | (None, 8, 8, 256) | 0      | ['batch_normalization_16[0][0]',<br>'batch_normalization_15[0][0]'] |
| re_lu_14 (ReLU)                              | (None, 8, 8, 256) | 0      | ['add_6[0][0]']                                                     |
| conv2d_17 (Conv2D)                           | (None, 8, 8, 256) | 590080 | ['re_lu_14[0][0]']                                                  |
| batch_normalization_17 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_17[0][0]']                                                 |
| re_lu_15 (ReLU)<br>7[0][0]']                 | (None, 8, 8, 256) | 0      | ['batch_normalization_17[0][0]']                                    |
| conv2d_18 (Conv2D)                           | (None, 8, 8, 256) | 590080 | ['re_lu_15[0][0]']                                                  |
| batch_normalization_18 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_18[0][0]']                                                 |
| add_7 (Add)<br>18[0][0]']                    | (None, 8, 8, 256) | 0      | ['re_lu_14[0][0]',<br>'batch_normalization_18[0][0]']               |
| re_lu_16 (ReLU)                              | (None, 8, 8, 256) | 0      | ['add_7[0][0]']                                                     |
| conv2d_19 (Conv2D)                           | (None, 8, 8, 256) | 590080 | ['re_lu_16[0][0]']                                                  |
| batch_normalization_19 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_19[0][0]']                                                 |
| re_lu_17 (ReLU)                              | (None, 8, 8, 256) | 0      | ['batch_normalization_19[0][0]']                                    |

|                                              |                   |        |                                                       |
|----------------------------------------------|-------------------|--------|-------------------------------------------------------|
| 9[0][0]']                                    |                   |        |                                                       |
| conv2d_20 (Conv2D)                           | (None, 8, 8, 256) | 590080 | ['re_lu_17[0][0]']                                    |
| batch_normalization_20 (Batch Normalization) | (None, 8, 8, 256) | 1024   | ['conv2d_20[0][0]']                                   |
| add_8 (Add)                                  | (None, 8, 8, 256) | 0      | ['re_lu_16[0][0]',<br>'batch_normalization_20[0][0]'] |
| re_lu_18 (ReLU)                              | (None, 8, 8, 256) | 0      | ['add_8[0][0]']                                       |
| average_pooling2d (AveragePooling2D)         | (None, 2, 2, 256) | 0      | ['re_lu_18[0][0]']                                    |
| flatten (Flatten)                            | (None, 1024)      | 0      | ['average_pooling2d[0][0]']                           |
| dense (Dense)                                | (None, 10)        | 10250  | ['flatten[0][0]']                                     |

=====

Total params: 4,344,842  
Trainable params: 4,338,570  
Non-trainable params: 6,272

None

The total parameters are 4.3M which are < 5M